
Bureau d'études Apprentissage automatique (machine learning)

Classification des fleurs

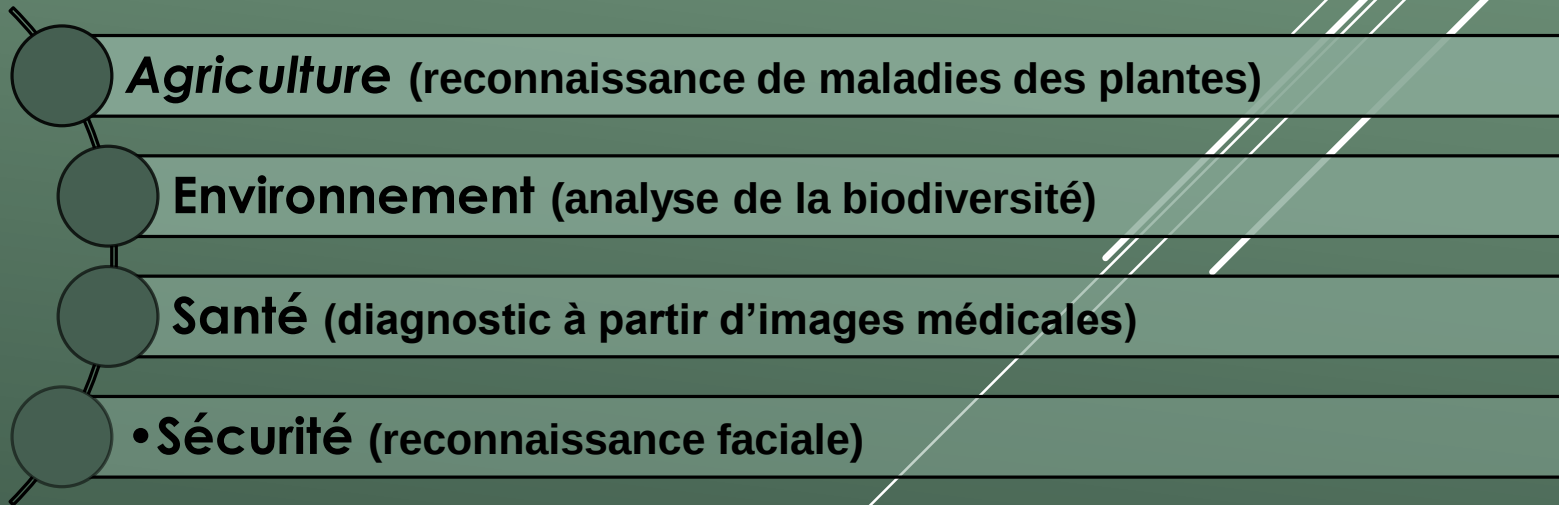
M1SIA2

- Réalisé par :
 - ✓ KABOU Abdeldjalil

13/05/2025

Classification

- Domaine fondamental de l'intelligence artificielle, notamment en apprentissage profond (Deep learning).
- Objectif : attribuer une étiquette ou une catégorie à une image selon son contenu visuel.
- Utilisée dans de nombreux domaines :



Classification des fleurs

- Classifier automatiquement des images de fleurs à partir de leurs photographies.
- Construire un modèle capable de :
 - ✓ Reconnaître visuellement l'espèce d'une fleur,
 - ✓ Attribuer la bonne catégorie parmi plus de 100 classes,
 - ✓ Se baser uniquement sur l'apparence visuelle (forme, couleur, texture).



Classification des fleurs

Les étapes principales du projet sont :

- Découpage des données en ensembles d'entraînement, validation et test,
- Construction d'un modèle CNN (réseau de neurones convolutionnel),
- Amélioration des performances par transfert de connaissances avec le modèle VGG16,
- Evaluation du modèle à l'aide de métriques classiques (précision, matrice de confusion),
- Enfin, la création d'une application web interactive avec Streamlit permettant de tester le modèle en téléversant une image.

Découpage des données

- Lire les images depuis le dossier Données
- Générer une nouvelle structure dans flower_output
- Les répartir automatiquement en 80 % train, 10 % validation, 10 % test

```
splitfolders.ratio(  
    r"F:\Université de Toulouse\Je suis en France\Cours\S8\BE Apprentissage automatique (Machine learning)\Projet\Donnees",  
    output= r"F:\Université de Toulouse\Je suis en France\Cours\S8\BE Apprentissage automatique (Machine learning)\Projet\flower_output",  
    seed=1337,  
    ratio=(.8, 0.1, 0.1)  
)
```

Nom

test
train
val

Découpage des données

- Définition des répertoires

```
train_dir = r"F:\Université de Toulouse\Je suis en France\Cours\S8\BE Apprentissage automatique (Machine learning)\Projet\flower_output\train"
val_dir = r"F:\Université de Toulouse\Je suis en France\Cours\S8\BE Apprentissage automatique (Machine learning)\Projet\flower_output\val"
test_dir = r"F:\Université de Toulouse\Je suis en France\Cours\S8\BE Apprentissage automatique (Machine learning)\Projet\flower_output\test"
```

- Création des Data Generators

```
train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(
    directory=train_dir,
    target_size=(180, 180),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32,
    shuffle=True,
    seed=2,
```

Générateur pour train

```
test_rescal=ImageDataGenerator(rescale=1./255)
test_gen=test_rescal.flow_from_directory(
    directory=test_dir,
    target_size=(180, 180),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32,
    shuffle=False,
    seed=None,
```

Générateur pour le test

```
val_datagen = ImageDataGenerator(rescale=1./255)
val_generator = val_datagen.flow_from_directory(
    directory=val_dir,
    target_size=(180, 180),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32,
    shuffle=True,
    seed=2,
```

Générateur pour validation

Découpage des données

- Création des Data Generators

```
train_datagen = ImageDataGenerator(rescale=1./255)
train_generator = train_datagen.flow_from_directory(
    directory=train_dir,
    target_size=(180, 180),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32,
    shuffle=True,
    seed=2,
```

Générateur pour train

```
test_rescal=ImageDataGenerator(rescale=1./255)
test_gen=test_rescal.flow_from_directory(
    directory=test_dir,
    target_size=(180, 180),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32,
    shuffle=False,
    seed=None,
```

Générateur pour le test

```
val_datagen = ImageDataGenerator(rescale=1./255)
val_generator = val_datagen.flow_from_directory(
    directory=val_dir,
    target_size=(180, 180),
    color_mode='rgb',
    class_mode='categorical',
    batch_size=32,
    shuffle=True,
    seed=2,
```

Générateur pour validation

```
Found 907 images belonging to 102 classes.
Found 6507 images belonging to 102 classes.
Found 775 images belonging to 102 classes.
```

Création du modèle CNN

- Première repère des bords et lignes
- Deuxième combine ces motifs pour former des formes
- Troisième détecte des textures et structures complètes utiles à la classification des fleurs.
- Couche Flatten pour transformer l'image en vecteur,

```
model = models.Sequential([  
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(180, 180, 3)),  
    layers.MaxPooling2D(2, 2),  
  
    layers.Conv2D(64, (3, 3), activation='relu'),  
    layers.MaxPooling2D(2, 2),  
  
    layers.Conv2D(128, (3, 3), activation='relu'),  
    layers.MaxPooling2D(2, 2),  
  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(train_generator.num_classes, activation='softmax')  
])
```


Création du modèle CNN

- MaxPooling2D pour réduire les dimensions des cartes de caractéristiques
(180×180 → 90×90 → 45×45)
- ReLU (Rectified Linear Unit) : Fonction d'activation qui permet de garder uniquement les activation positive
- la dernière Dense utilise une softmax pour produire une probabilité pour chaque classe.

```
model = models.Sequential([  
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(180, 180, 3)),  
    layers.MaxPooling2D(2, 2),  
  
    layers.Conv2D(64, (3, 3), activation='relu'),  
    layers.MaxPooling2D(2, 2),  
  
    layers.Conv2D(128, (3, 3), activation='relu'),  
    layers.MaxPooling2D(2, 2),  
  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(train_generator.num_classes, activation='softmax')  
])
```

Compilation du modèle

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

- Categorical Cross entropy (log-loss)
- Le % d'images bien classées

Entraînement du modèle

```
model.fit(  
    train_generator,  
    validation_data=val_generator,  
    epochs=10  
)
```

- La fonction model.fit() entraîne le modèle lui montrant les images plusieurs fois
- Le modèle est entraîné pendant 10 époques sur les données d'entraînement

Evaluation du modèle

- Evaluation du modèle sur le jeu de test (les images de test que le modèle n'a jamais vues avant) en utilisant `model.evaluate()`

```
test_loss, test_acc = model.evaluate(test_gen)
print('Test accuracy:', test_acc)
```

Analyse des résultats

- CNN simple, nous avons obtenu une précision de seulement 35 % sur les données de test, ce qui indique une faible capacité de généralisation.

```
204/204 ————— 124s 607ms/step
0.0482 - val_accuracy: 0.3548 - val_loss: 4
Epoch 10/10
204/204 ————— 124s 610ms/step
0.0583 - val_accuracy: 0.3445 - val_loss: 5
29/29 ————— 15s 513ms/step -
Test accuracy: 0.3550165295600891
```

Data augmentation

- A l'aide du module ImageDataGenerator de Keras sur les images d'entraînement.

Malheureusement !!

- Malgré l'ajout de data augmentation, les performances du modèle CNN simple sont restées insuffisantes,
- Avec une précision en test 33 % même inférieure à la version de base

```
# Data Augmentation (Avec les données fictives)

train_datagen = ImageDataGenerator(
    rescale=1./255,
    width_shift_range=0.01,
    height_shift_range=0.01,
    horizontal_flip=True,
    rotation_range=10,
    brightness_range=[0.15,2.0],
    zoom_range=[0.99,0.01]
)
```

```
or `keras.saving.save_model(model)`.
We recommend using instead the native
`model.save('my_model.keras')` or `ke
'my_model.keras')`.
Test accuracy: 0.33186328411102295
```

Transfer Learning

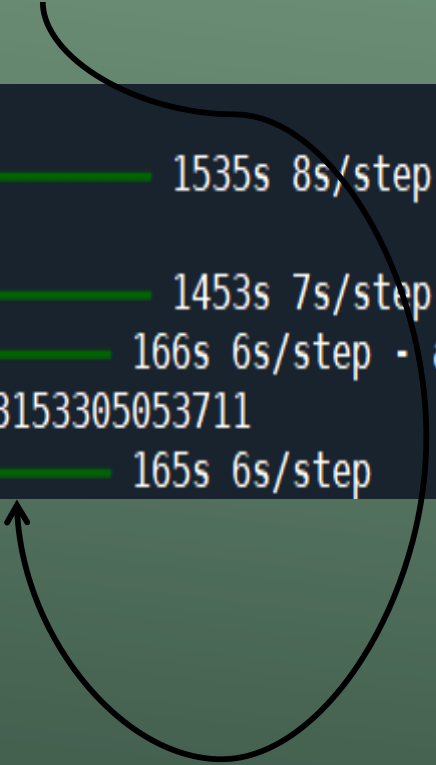
- Utilisation des réseaux pré-entraînés
- Pour améliorer significativement la capacité du modèle à généraliser, nous allons alors adopter une stratégie de Transfer Learning.
- Cela consiste à utiliser un réseau pré-entraîné VGG16 (il est déjà entraîné sur ImageNet) comme extracteur de caractéristiques.
- Nous avons gelé les 17 premières couches du modèle VGG16 pour conserver les filtres appris,
- Nous avons ajouté nos propres couches Dense pour adapter le modèle à notre base de fleurs.

VGG-16
PRETRAINED MODEL

Transfer Learning

- Modèle VGG16 a atteint une précision (accuracy) de 70 % (sur 10 epoch) sur les données de test.

```
Epoch 9/10
204/204 _____ 1535s 8s/step -
Epoch 10/10
204/204 _____ 1453s 7s/step -
29/29 _____ 166s 6s/step - acc
Test accuracy: 0.7023153305053711
29/29 _____ 165s 6s/step
```



```
num_classes=102
base_model = VGG16(include_top=False, weights='imagenet', input_shape=(224, 224, 3))
# Les paramètres des 17 premières couches sont gelés
for layer in base_model.layers[:17]:
    layer.trainable = False
# Pour voir quelles sont les couches entraînables
for i, layer in enumerate(base_model.layers):
    print(i, layer.name, layer.trainable)
model = models.Sequential([
    base_model,
    layers.Flatten(),
    layers.Dense(1024, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(512, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(num_classes, activation='softmax')
])
```

Compilation du modèle

```
model.compile(optimizer='adam',  
              loss='categorical_crossentropy',  
              metrics=['accuracy'])
```

- Même chose que le premier programme : Adam + cross-entropy pour multi-classes.

Entraînement du modèle

```
model.fit(  
    train_generator,  
    validation_data=val_generator,  
    epochs=10  
)
```

- Le modèle apprend sur train_generator, et se valide à chaque époque sur val_generator.

Evaluation du modèle

- Evaluation du modèle sur le jeu de test (les images de test que le modèle n'a jamais vues avant) en utilisant `model.evaluate()`

```
test_loss, test_acc = model.evaluate(test_gen)
print('Test accuracy:', test_acc)
```

Sauvegarder le modèle

- Sauvegarder le modèle pour le réutiliser plus tard
- Contient l'architecture du modèle (type de couches, activation, dimensions, etc.)
- Contient les poids appris après l'entraînement.

```
model_json = model.to_json()
with open('model_flower.json', 'w') as json_file:
    json_file.write(model_json)
model.save_weights('model_flower.weights.h5')
```



model_flower.json



model_flower.weights.h5

Analyse des résultats

- Obtenir les prédictions du modèle sur les données de test.
- Comparer ces prédictions aux étiquettes vraies.
- Afficher :

- Une matrice de confusion

```
[[2 0 0 ... 0 0 0]
 [0 3 0 ... 1 0 0]
 [0 0 1 ... 0 1 0]
 ...
 [0 0 0 ... 6 0 0]
 [0 0 0 ... 0 7 0]
 [0 0 0 ... 0 0 5]]
```

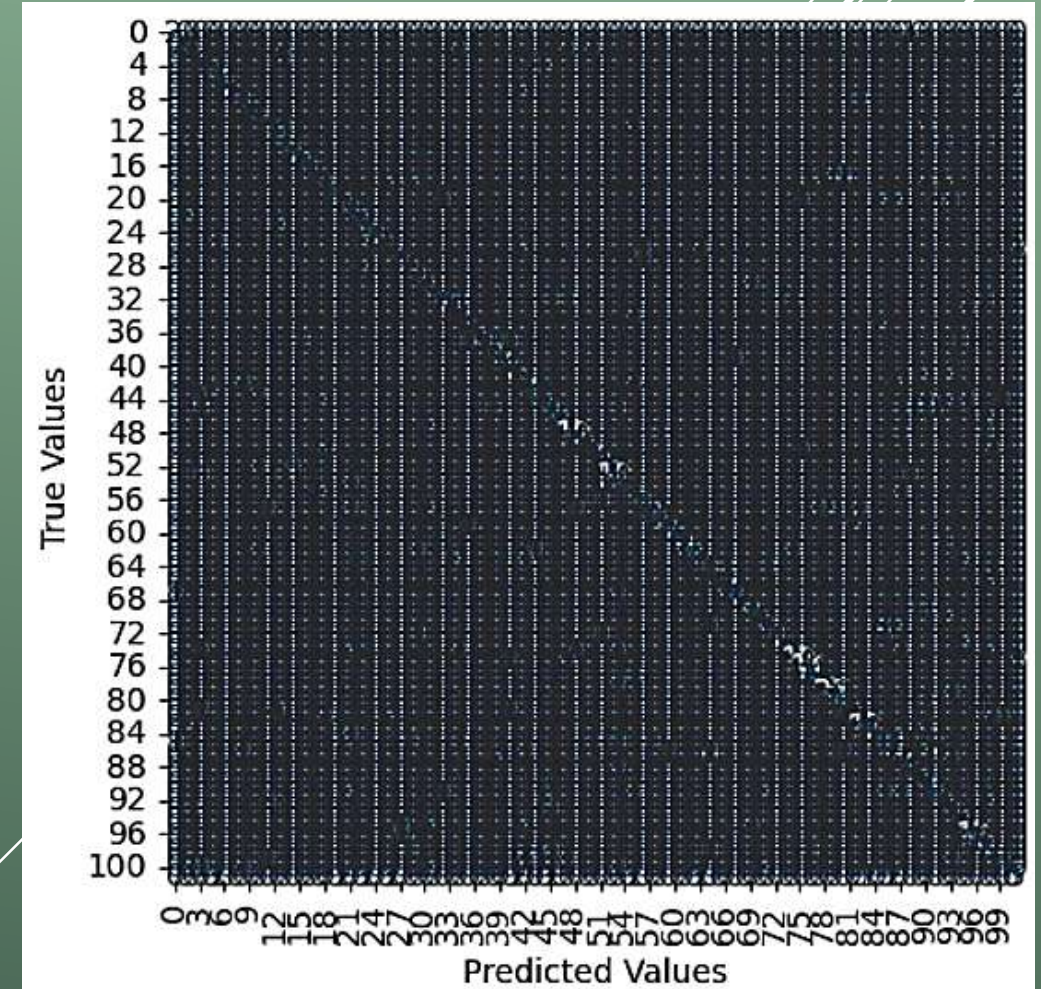
- Un rapport de classification

	precision	recall	f1-score	support
0	0.67	0.50	0.57	4
1	1.00	0.50	0.67	6
2	0.17	0.25	0.20	4
3	0.00	0.00	0.00	7
4	0.67	0.86	0.75	7

100	0.47	0.78	0.58	9
101	0.33	0.71	0.45	7
accuracy			0.70	907
macro avg	0.70	0.67	0.66	907
weighted avg	0.73	0.70	0.70	907

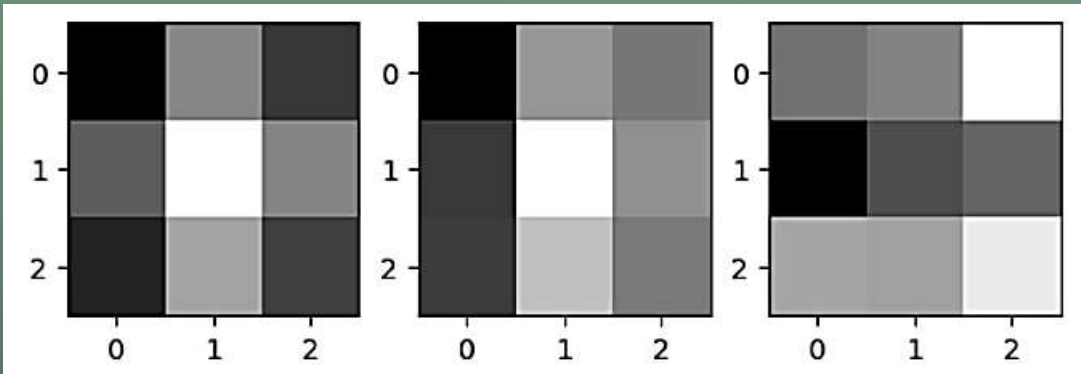
Analyse des résultats

- Obtenir les prédictions du modèle sur les données de test.
- Comparer ces prédictions aux étiquettes vraies.
- Afficher :
 - Une visualisation (heatmap) claire des erreurs.
 - La diagonale montre les bonnes prédictions.
 - Hors diagonale = erreurs de classification.



Visualisation des filtres et activations dans VGG16

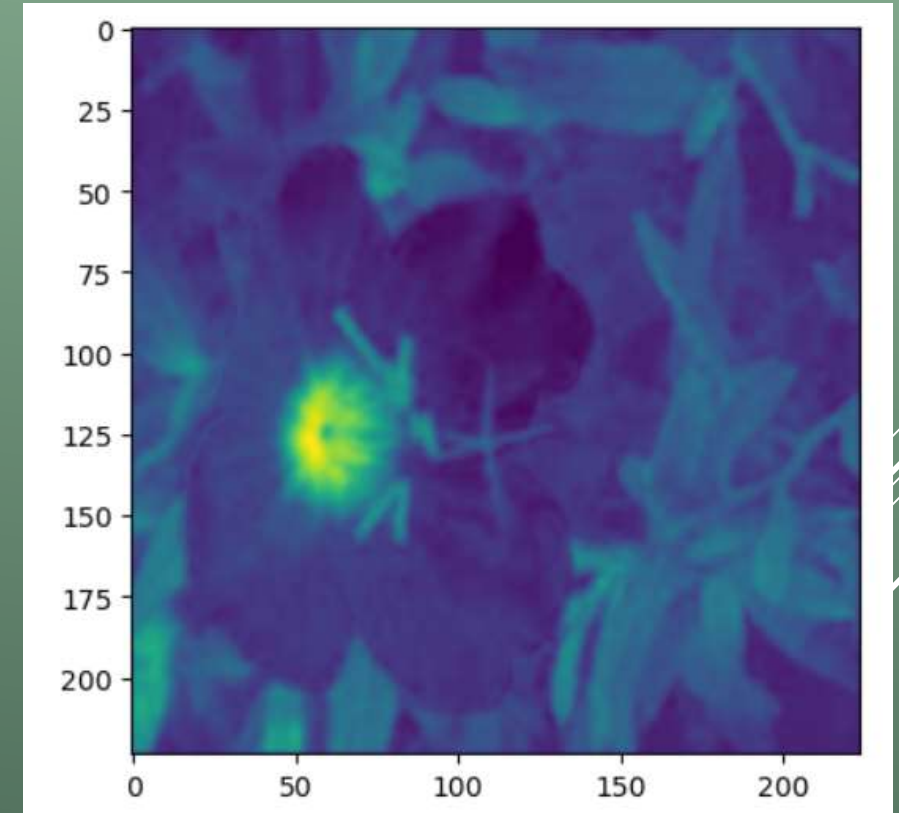
- Prendre le deuxième filtre de la première couche convolutionnelle (qui contient 3 sous-filtres pour les 3 canaux R, V et B) et visualiser ces 3 sous-filtres



- Visualiser les feature maps produites par la deuxième couche
- Prédiction finale

```
proba=model.predict(image)
pred_classe=np.argmax(proba, 1)
print("Classe =", pred_classe)
```

1/1 ————— 1s 564ms/step
Classe = [0]



C-à-d la classe 1

Visualisation des filtres et activations dans VGG16

- Prédiction finale

```
output/test/flower_02/image_05110.jpg'
Total params: 1,792 (7.00 KB)
Trainable params: 0 (0.00 B)
Non-trainable params: 1,792 (7.00 KB)
1/1 _____ 0s 60ms/step
1/1 _____ 1s 510ms/step
classe = [1]
```

C-à-d la classe 2

```
er_output/test/flower_05/image_05168.jp
Total params: 1,792 (7.00 KB)
Trainable params: 0 (0.00 B)
Non-trainable params: 1,792 (7.00 KB)
1/1 _____ 0s 70ms/step
1/1 _____ 1s 670ms/step
classe = [4]
In [39]:
```

C-à-d la classe 5

La création d'une API

- Dans cette dernière étape du projet, nous avons développé une interface utilisateur avec Streamlit
- Permettant de téléverser une image de fleur et d'obtenir automatiquement sa classe prédite (C'est une liste de 102 noms de fleurs. Chaque index (0, 1, 2, ...) correspond à une classe prédite par le modèle.).
- Le nom de la fleur prédite est ensuite affiché dynamiquement dans l'interface.

Interface utilisateur pour la classification d'images

- Pour lancer l'application web (API → Anaconda Prompt → `streamlit run BE_partie_6.py`)
- Une fois l'application ouverte dans le navigateur, il est possible de téléverser n'importe quelle image de fleur, et le modèle pré-entraîné affichera automatiquement :
 - ✓ la classe prédite (nom de la fleur),

Conclusion

Merci



Sources et Références

- Ce projet a été réalisé à partir des fichiers et supports fournis dans les Travaux Pratiques,

